

Dynamic Programming 2(D)



DNA Comparison:

Longest Common Subsequence (LCS)

The length of the longest common subsequence (LCS) between two genomes is a measure of similarity.

S1 ACCGGTCGAGTGCGCGGAAGCCGGCCGAA

S2 GTCGTTCGGAATGCCGTTGCTCTGTAA

What is an LCS of
“Go Blue” and “Ghoul”?



LCS Recurrence (prof. Y's solution)

Given: 2 strings S_1 and S_2 of length N

Consider an LCS, call it s .

Consider the last symbols of S_1 and S_2

Let $\text{LCS}(S_1, S_2)$ be the length of an LCS of S_1 and S_2

Case 1, use it: s uses the last symbols of S_1 and S_2

$$\text{LCS}(S_1[1..N], S_2[1..N]) = \text{LCS}(S_1[1..N-1], S_2[1..N-1]) + 1$$

Case 2, lose it: s does not use the last symbols of S_1 and S_2

$$\text{LCS}(S_1[1..N], S_2[1..N]) = \text{LCS}(S_1[1..N-1], S_2[1..N-1])$$

Counterexample:

ABCDEF

CDEFGH



LCS Recurrence

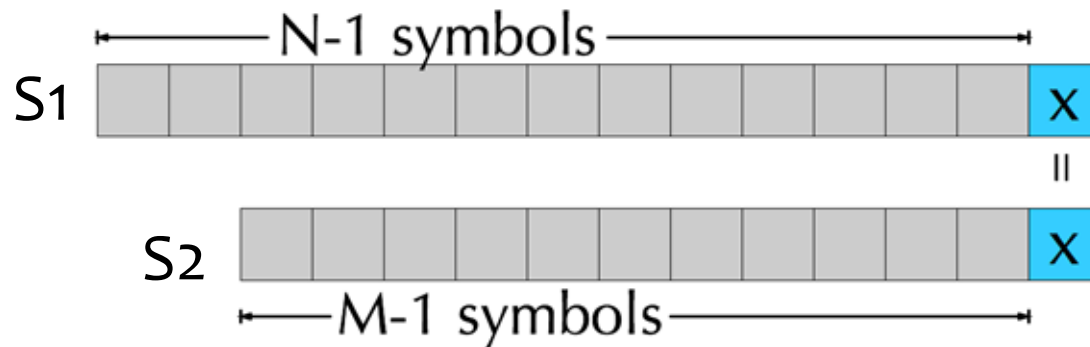
Given: 2 strings S_1 of length N and S_2 of length M

Consider an LCS, call it s .

Consider the last symbols of S_1 and S_2

Case 1, use it: s uses the last symbols of S_1 and S_2

$$\text{LCS}(S_1[1..N], S_2[1..M]) = \text{LCS}(S_1[1..N-1], S_2[1..M-1]) + 1$$



Case 2, lose it: s does not use at least one of $S_1[N]$ and $S_2[M]$

LCS Recurrence

Given: 2 strings S_1 of length N and S_2 of length M

Consider an LCS, call it s .

Consider the last symbols of S_1 and S_2

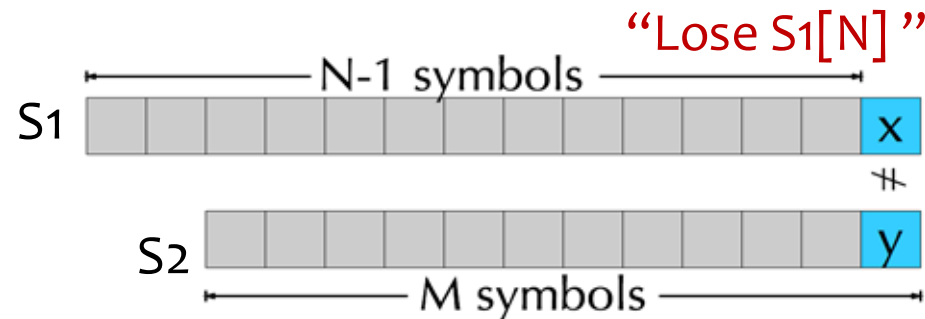
Case 1, use it: s uses the last symbols of S_1 and S_2

$$\text{LCS}(S_1[1..N], S_2[1..M]) = \text{LCS}(S_1[1..N-1], S_2[1..M-1]) + 1$$

Case 2, lose it: s does not use at least one of $S_1[N]$ and $S_2[M]$

Case 2a: s does not use $S_1[N]$

$$\text{LCS}(S_1[1..N-1], S_2[1..M])$$



Case 2b: s does not use $S_2[M]$

$$\text{LCS}(S_1[1..N], S_2[1..M-1])$$

Full Recurrence for LCS (version 1)

$$\text{LCS}(S_1[1..N], S_2[1..M]) =$$

$$\max \left(\begin{array}{ll} \text{LCS}(S_1[1..N-1], S_2[1..M-1]) + 1 & \text{if } S_1[N] = S_2[M] \text{ (use it)} \\ \text{LCS}(S_1[1..N-1], S_2[1..M]) & \text{(lose } S_1[N]) \\ \text{LCS}(S_1[1..N], S_2[1..M-1]) & \text{(lose } S_2[M]) \end{array} \right)$$

In fact, if $S_1[N] = S_2[M]$, we don't need the other cases!

Any LCS using only 1 could be switched to use both

Any LCS using neither could be lengthened by adding them (not actually longest)

Full Recurrence for LCS (version 2)

$$\text{LCS}(S_1[1..N], S_2[1..M]) =$$

$$\begin{cases} \text{LCS}(S_1[1..N-1], S_2[1..M-1]) + 1 & \text{if } S_1[N] = S_2[M] \quad (\text{use it}) \\ \max \begin{pmatrix} \text{LCS}(S_1[1..N-1], S_2[1..M]) \\ \text{LCS}(S_1[1..N], S_2[1..M-1]) \end{pmatrix} & \text{otherwise} \end{cases}$$

(lose $S_1[N]$)
(lose $S_2[M]$)

ε is a string, like "abcdef",
but it has 0 characters. $\varepsilon = ""$

Base cases: $\text{LCS}(S_1[1..i], \varepsilon) = 0$ for all i

$\text{LCS}(\varepsilon, S_2[1..j]) = 0$ for all j

Let's Follow the DP Recipe

1. define value func
2. find recurrence
3. size of table
4. base cases
5. fill order
6. final answer
7. size*time
8. reconstruction

$$\text{LCS}(S_1[1..N], S_2[1..M]) = \begin{cases} \text{LCS}(S_1[1..N-1], S_2[1..M-1]) + 1 & \text{if } S_1[N] = S_2[M] \\ \max \begin{pmatrix} \text{LCS}(S_1[1..N-1], S_2[1..M]) \\ \text{LCS}(S_1[1..N], S_2[1..M-1]) \end{pmatrix} & \text{otherwise} \end{cases}$$

$S_1 = \text{GAC}; S_2 = \text{AGCAT}$

	ϵ	...A	...G	...C	...A	...T
ϵ						
...G						
...A						
...C						

1. define value func
2. find recurrence
3. size of table
4. base cases
5. fill order
6. final answer
7. size*time
8. reconstruction

1. define value func
2. find recurrence
3. size of table
4. base cases
5. fill order
6. final answer
7. size*time
8. reconstruction

1. define value func
2. find recurrence
3. size of table
4. base cases
5. fill order
6. final answer
7. size*time
8. reconstruction

Let's Follow the DP Recipe

1. define value func
2. find recurrence
3. size of table
4. base cases
5. fill order
6. final answer
7. size*time
8. reconstruction

$$\text{LCS}(S_1[1..N], S_2[1..M]) = \begin{cases} \text{LCS}(S_1[1..N-1], S_2[1..M-1]) + 1 & \text{if } S_1[N] = S_2[M] \\ \max \begin{pmatrix} \text{LCS}(S_1[1..N-1], S_2[1..M]) \\ \text{LCS}(S_1[1..N], S_2[1..M-1]) \end{pmatrix} & \text{otherwise} \end{cases}$$

$S_1 = \text{GAC}; S_2 = \text{AGCAT}$ Runtime: $(N + 1)(M + 1) \cdot O(1) = O(NM)$

	ϵ	...A	...G	...C	...A	...T
ϵ	0	0	0	0	0	0
...G	0	0	1	1	1	1
...A	0	1	1	1	2	2
...C	0	1	1	2	2	2

Subsequence: GA

Let's Follow the DP Recipe

1. define value func
2. find recurrence
3. size of table
4. base cases
5. fill order
6. final answer
7. size*time
8. reconstruction

$$\text{LCS}(S_1[1..N], S_2[1..M]) = \begin{cases} \text{LCS}(S_1[1..N-1], S_2[1..M-1]) + 1 & \text{if } S_1[N] = S_2[M] \\ \max \begin{pmatrix} \text{LCS}(S_1[1..N-1], S_2[1..M]) \\ \text{LCS}(S_1[1..N], S_2[1..M-1]) \end{pmatrix} & \text{otherwise} \end{cases}$$

$S_1=\text{GAC}; S_2=\text{AGCAT}$			Can we still reconstruct a solution without breadcrumbs?				
	ϵ	...A	...G	...C	...A	...T	
ϵ	0	0	0	0	0	0	
...G	0	0	1	1	1	1	
...A	0	1	1	1	2	2	
...C	0	1	1	2	2	2	

Let's Follow the DP Recipe

1. define value func
2. find recurrence
3. size of table
4. base cases
5. fill order
6. final answer
7. size*time
8. reconstruction

$$\text{LCS}(S_1[1..N], S_2[1..M]) = \begin{cases} \text{LCS}(S_1[1..N-1], S_2[1..M-1]) + 1 & \text{if } S_1[N] = S_2[M] \\ \max \begin{pmatrix} \text{LCS}(S_1[1..N-1], S_2[1..M]) \\ \text{LCS}(S_1[1..N], S_2[1..M-1]) \end{pmatrix} & \text{otherwise} \end{cases}$$

$S_1 = \text{GAC}; S_2 = \text{AGCAT}$		Can we still reconstruct a solution without breadcrumbs?				
	ϵ	...A	...G	...C	...A	...T
ϵ	0	0	0	0	0	0
...G	0	0	1	1	1	1
...A	0	1	1	1	2	2
...C	0	1	1	2	2	2

Subsequence: AC

Pseudocode

Algorithm LCS($S_1[1..N]$, $S_2[1..M]$)

table := 2D-array indexed from 0 to N, and 0 to M

for $i = 0$ to N:

table($i, 0$) = 0

for $i = 0$ to M:

table(0, i) = 0

for $i = 1$ to N:

for $j = 1$ to M:

if $S_1[i] = S_2[j]$:

table(i, j) = 1 + **table**($i-1, j-1$)

else:

table(i, j) = $\max\{\mathbf{table}(i-1, j), \mathbf{table}(i, j-1)\}$

Return table(N,M)

A Visualization of LCS DP

<https://www.cs.usfca.edu/~galles/visualization/DPLCS.html>

Can we do better than $O(MN)$ time?

Under a conjecture called the “strong exponential time hypothesis,” *there is no algorithm* with an $O(MN^{0.999})$ running time!

If the hypothesis is true, you’ve seen the (essentially) optimal algorithm.

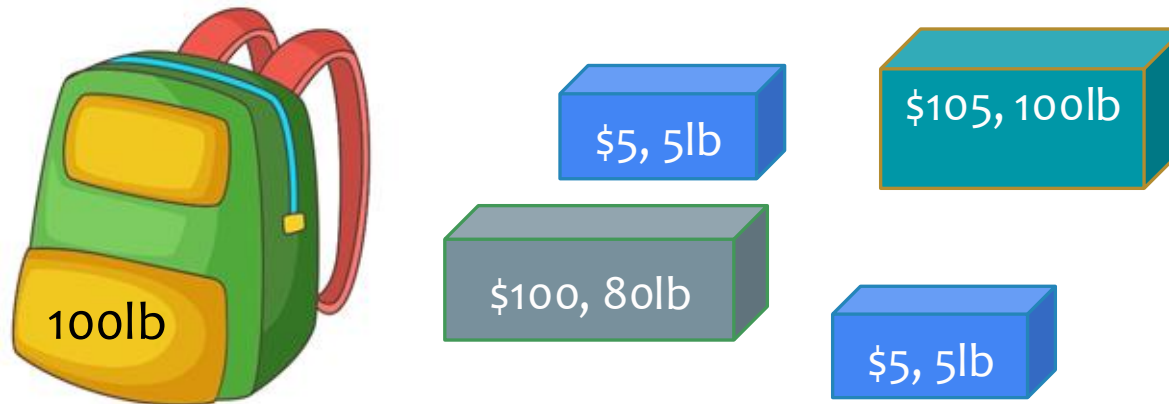
But... |human genome| $\approx$ 3bil, |chimp genome| $\approx$ 2.8bil

– **Current research:** faster approximation algorithms

Knapsack Problem

Input: A set of n items $t_1, \dots, t_n$, each $t_i = (\text{value } v_i, \text{integer weight } w_i)$, along with an integer W = the total “weight capacity” of the knapsack.

Output: A set of items whose *total weight is at most W* and whose *total value is maximized*.



Question: What is the input size in terms of n and W ? $n \log(W)$

(Suppose each individual v_i is small enough that we can disregard its contribution to the input size, but each w_i could be up to W)

Knapsack Recurrence

Define $\text{Knapsack}(T, W)$ = optimum *value* for items T , with limit W .

Consider item t_n .

$$\text{Knapsack}(\{t_1, \dots, t_n\}, W) = \max \left(\begin{array}{ll} \text{Knapsack}(\{t_1, \dots, t_{n-1}\}, W - w_n) + v_n & \text{if } w_n \leq W \quad \text{(use it!)} \\ \text{Knapsack}(\{t_1, \dots, t_{n-1}\}, W) & \quad \text{(lose it!)} \end{array} \right)$$

Base cases: $\text{Knapsack}(\{\}, W) = 0$

Let's follow the DP Recipe

1. define value func
2. find recurrence
3. size of table
4. base cases
5. fill order
6. final answer
7. size*time
8. reconstruction

$$\text{Knapsack}(\{t_1, \dots, t_n\}, W) = \max \begin{pmatrix} \text{Knapsack}(\{t_1, \dots, t_{n-1}\}, W - w_n) + v_n & \text{if } w_n \leq W \\ \text{Knapsack}(\{t_1, \dots, t_{n-1}\}, W) \end{pmatrix}$$

remaining weight

items up through

	0	1	2	3
$\emptyset$				
t1: 4\$, 2lb				
t2: 8\$, 1lb				
t3: 6\$, 2lb				
t4: 5\$, 1lb				

Let's follow the DP Recipe

1. define value func
2. find recurrence
3. size of table
4. base cases
5. fill order
6. final answer
7. size*time
8. reconstruction

$$\text{Knapsack}(\{t_1, \dots, t_n\}, W) = \max \begin{pmatrix} \text{Knapsack}(\{t_1, \dots, t_{n-1}\}, W - w_n) + v_n & \text{if } w_n \leq W \\ \text{Knapsack}(\{t_1, \dots, t_{n-1}\}, W) \end{pmatrix}$$

remaining weight

items up through

	0	1	2	3
$\emptyset$	0	0	0	0
t ₁ : 4\$, 2lb	0	0	4	4
t ₂ : 8\$, 1lb	0	8	8	12
t ₃ : 6\$, 2lb	0	8	8	14
t ₄ : 5\$, 1lb	0	8	13	14

Let's follow the DP Recipe

1. define value func
2. find recurrence
3. size of table
4. base cases
5. fill order
6. final answer
7. size*time
8. reconstruction

$$\text{Knapsack}(\{t_1, \dots, t_n\}, W) = \max \begin{pmatrix} \text{Knapsack}(\{t_1, \dots, t_{n-1}\}, W - w_n) + v_n & \text{if } w_n \leq W \\ \text{Knapsack}(\{t_1, \dots, t_{n-1}\}, W) \end{pmatrix}$$

Runtime: $(n+1)(W+1) \cdot O(1) = O(nW)$

remaining weight

	0	1	2	3
$\emptyset$	0	0	0	0
t₁ : 4\$, 2lb	0	0	4	4
t₂ : 8\$, 1lb	0	8	8	12
t₃ : 6\$, 2lb	0	8	8	14
t₄ : 5\$, 1lb	0	8	13	14

items up through

Items: T_2, T_3

Pseudocode

Algorithm Knapsack($\{t_1, \dots, t_n\}$, W)

table := 2D-array indexed from 0 to n , and 0 to W

for $j = 0$ to W : **table**(0, j) = 0

for $i = 0$ to n :

 for $j = 0$ to W :

 if $w_i \leq j$:

table(i, j) = $\max(v_i + \text{table}(i-1, j - w_i), \text{table}(i-1, j))$

 else:

table(i, j) = **table**($i-1, j$)

Return table(n, W)

What is the running time, in terms of input length?

Other “>1-Dimensional” DP Problems

1. Max-sum submatrix of a given matrix
2. Optimum binary search tree (for known access frequencies)
3. Viterbi algorithm (speech recognition, error correction, ...)
4. Shortest paths in graphs: Next time
5. And many more...

Next Time: DP on Graphs

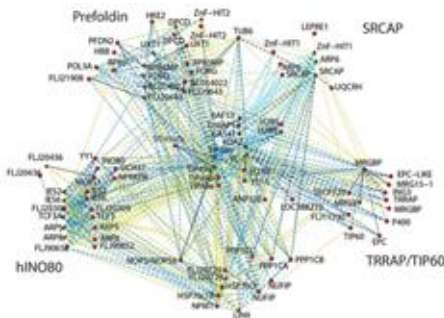
Graphs can represent many things!



The Internet



Social Networks



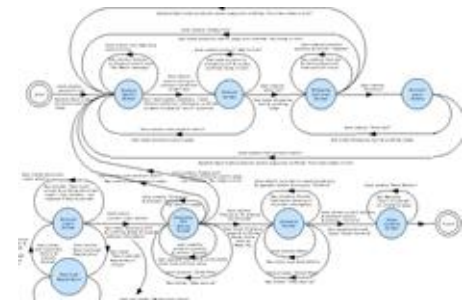
Biological and Chemical Networks



Circuits



Transportation Networks



State Transition Networks